

HackNEX — Registration & Payment Technical Design Document

Document Version: 1.0

Project: HackNEX

Organizer: NEXUS Club, Karunya Institute of Technology and Sciences

Event: October 7–9, 2026

Registration Period: September 8 – October 1, 2026

Event Mode: Offline

Expected Participants: 1,500+

Team Size: Exactly 4 participants

Registration Fee: ₹600 per team

Payment Provider: Karunya payment mechanism — integration details TBD

Refund Policy: No refund

Backend: Fastify + TypeScript

ORM: Prisma

Database: PostgreSQL

Authentication: Email + Password + Email Verification

Email: Resend

1. Purpose

This document defines the complete technical design for the **HackNEX team registration and payment system**.

The system must allow an authenticated participant to:

1. Create a HackNEX account.
2. Verify their email.
3. Login.
4. Register a team.
5. Enter details for exactly 4 participants.
6. Select food preference for each participant.
7. Submit the registration.
8. Pay the ₹600 registration fee through the Karunya payment mechanism.
9. Have the payment automatically verified.
10. Receive the registration-success email **only after successful payment verification**.

The system must also prevent:

- Duplicate participants

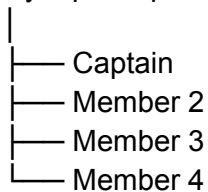
- Duplicate team names
 - Duplicate team registrations by a captain
 - Registration without authentication
 - Registration without email verification
 - Invalid team sizes
 - Payment amount manipulation
 - False payment confirmation
 - Confirmation emails before successful payment
-

2. Core Business Rules

These rules are authoritative.

Team

Exactly 4 participants



Captain

The authenticated user who creates the registration becomes the team captain.

A captain cannot create multiple teams.

Participants

A team may contain participants from:

- Different colleges
- Different departments
- Different years of study

Participant uniqueness

A participant cannot be registered in multiple teams.

Team name

Team names must be unique.

Payment

₹600 per team

Refund

NO REFUND

Confirmation

Registration is considered successful **only after payment has been automatically verified.**

3. Registration Eligibility

A user can begin team registration only when:

Authenticated

+

Email Verified

+

Registration Period Active

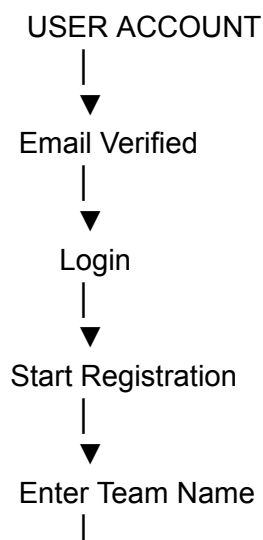
+

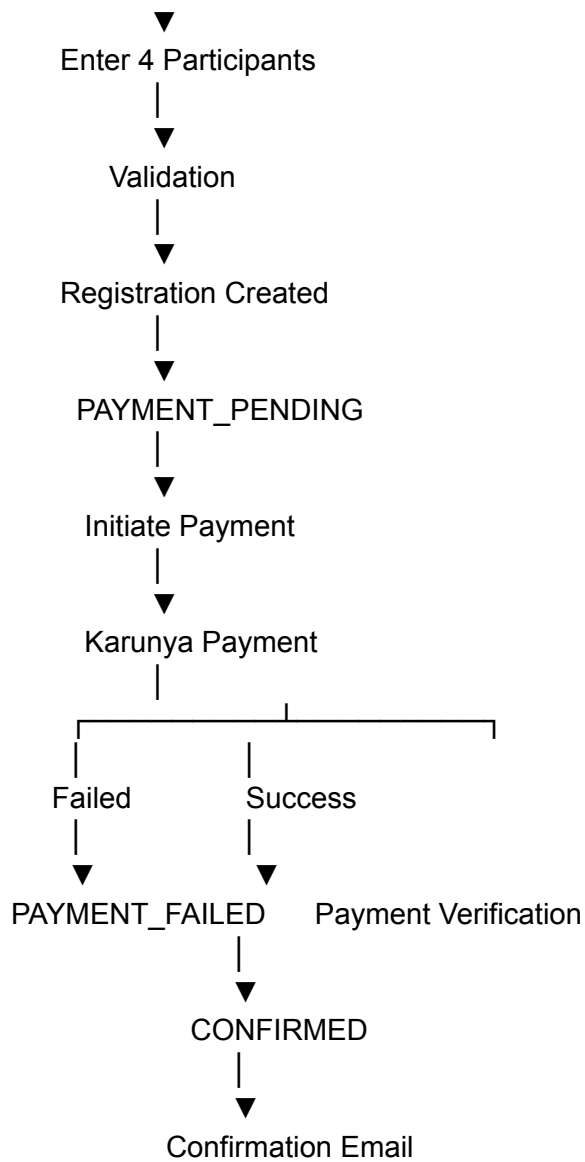
Captain Has No Existing Team

All four conditions must be satisfied.

4. Complete Registration Lifecycle

The overall lifecycle is:





5. Registration Data

Each participant requires:

Field	Required
Name	Yes
Email	Yes
Phone Number	Yes
College	Yes
Department	Yes

Year of Study	Yes
LinkedIn URL	Yes
Food Preference	Yes

Food preference:

VEG
NON_VEG

The captain selects the food preference for **all four participants**, including themselves.

6. Team Information

The captain provides:

Team Name

The backend generates:

Team ID
Registration ID

The user must not manually provide these identifiers.

7. Registration Form Architecture

The frontend registration page should be structured as:

Step 1
Team Details
↓
Step 2
Captain Details
↓
Step 3
Member 2
↓
Step 4
Member 3

↓
Step 5
Member 4
↓
Step 6
Review
↓
Step 7
Payment

Alternatively, the four participant forms can appear on one page with sections.

The backend treats the submission as **one atomic registration operation**.

8. Captain Details

The captain is already authenticated.

Therefore, the system should pre-populate:

Captain Name
Captain Email

from the authenticated account where appropriate.

The captain still provides the remaining registration information:

Phone
College
Department
Year
LinkedIn
Food Preference

9. Member Details

For each additional participant:

Name
Email
Phone Number
College
Department

Year of Study
LinkedIn
Food Preference

10. Cross-College Teams

Mixed-college teams are allowed.

Example:

Captain → Karunya
Member 2 → PSG
Member 3 → CIT
Member 4 → Other College

This is valid.

11. Cross-Department Teams

Mixed-department teams are allowed.

Example:

CSE
ECE
AI&DS
Mechanical

can all belong to the same team.

12. Registration Period

The official registration period is:

START
September 8, 2026

END
October 1, 2026

The backend must enforce this.

The frontend countdown or registration button does **not** determine whether registration is open.

13. Registration Deadline Logic

Every registration-related mutation must check:

Current Server Time

↓

Registration Start?

↓

Registration End?

If outside the registration window:

REGISTRATION_CLOSED

must be returned.

14. Registration Creation API

POST /api/v1/registrations

Authentication:

Required

Email verification:

Required

15. Registration Request

Example:

```
{
  "teamName": "Code Titans",
  "participants": [
```



```
{
  "name": "Captain Name",
  "email": "captain@example.com",
  "phone": "9876543210",
  "college": "Karunya Institute",
  "department": "CSE",
  "yearOfStudy": "2",
  "linkedinUrl": "https://linkedin.com/in/captain",
  "foodPreference": "VEG"
},
{
  "name": "Member Two",
  "email": "member2@example.com",
  "phone": "9876543211",
  "college": "PSG College",
  "department": "ECE",
  "yearOfStudy": "3",
  "linkedinUrl": "https://linkedin.com/in/member2",
  "foodPreference": "NON_VEG"
},
{
  "name": "Member Three",
  "email": "member3@example.com",
  "phone": "9876543212",
  "college": "CIT",
  "department": "IT",
  "yearOfStudy": "2",
  "linkedinUrl": "https://linkedin.com/in/member3",
  "foodPreference": "VEG"
},
{
  "name": "Member Four",
  "email": "member4@example.com",
  "phone": "9876543213",
  "college": "Other College",
  "department": "CSE",
  "yearOfStudy": "1",
  "linkedinUrl": "https://linkedin.com/in/member4",
  "foodPreference": "NON_VEG"
}
]
```

16. Backend Validation

The backend must validate:

Team

Team name exists

Team name length

Team name uniqueness

Participants

Exactly 4 participants

No duplicate emails within team

No participant already registered elsewhere

All required fields present

Valid email

Valid phone

Valid LinkedIn URL

Valid year

Valid food preference

Captain

Authenticated user

Verified email

Not already captain

17. Duplicate Participant Protection

The most important uniqueness rule:

A participant's email cannot appear in another HackNEX team.

Example:

Team A

infant@example.com

Then:

Team B

infant@example.com

must be rejected.

The database should enforce this, not only the frontend.

18. Team Name Uniqueness

Team names must be unique.

Example:

Team A → Code Titans

Another team attempting:

Code Titans

receives:

409 CONFLICT

Recommended normalization:

"Code Titans"

" code titans "

"CODE TITANS"

should be treated as the same logical team name.

19. Registration Transaction

Creating a registration involves multiple database operations.

These must occur inside a database transaction:

BEGIN TRANSACTION

Create Team



Create Participants



Create Registration



Create Payment



COMMIT

If any operation fails:

ROLLBACK

This prevents partially created teams.

20. Initial Registration State

After successful registration creation:

Registration

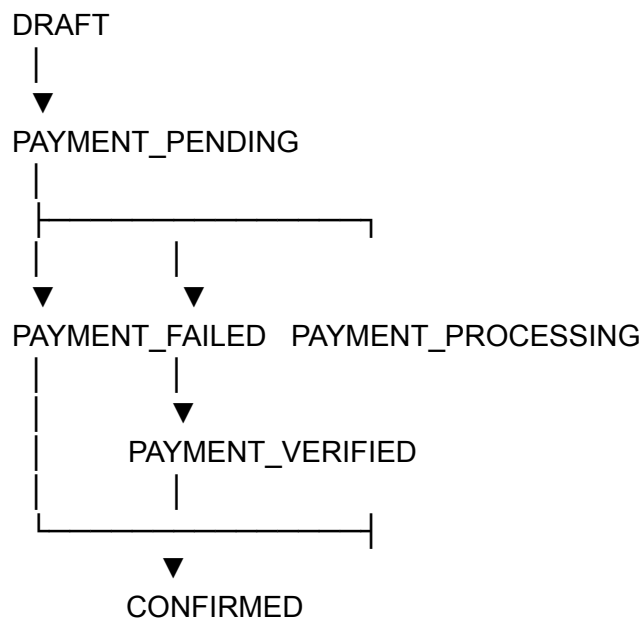


PAYMENT_PENDING

The team is **not confirmed yet**.

21. Registration State Machine

Recommended states:

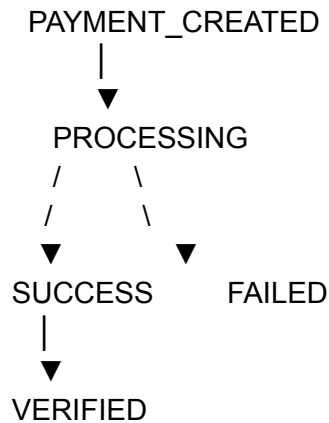


For operational simplicity, the persistent registration states can be:

DRAFT
PAYMENT_PENDING
CONFIRMED
CANCELLED

Payment has its own state machine.

22. Payment State Machine



Recommended enum:

```
PENDING
PROCESSING
SUCCESS
FAILED
VERIFIED
```

VERIFIED is the state that authorizes final registration confirmation.

23. Why Registration and Payment States Are Separate

Do not use one status field for everything.

Instead:

```
Registration
status = PAYMENT_PENDING
```

```
Payment
status = PROCESSING
```

Later:

```
Registration
status = CONFIRMED
```

Payment
status = VERIFIED

This gives administrators a clear picture of what happened.

24. Payment Amount

The registration fee is:

₹600

The frontend must **never be allowed to decide the amount**.

Bad:

```
{  
  "registrationId": "...",  
  "amount": 1  
}
```

The backend calculates:

REGISTRATION_FEE = 600

and sends that amount to the payment provider.

25. Currency

INR

The backend controls:

amount = 600
currency = INR

26. Payment Creation

Endpoint:

POST /api/v1/payments/create

Request:

```
{  
  "registrationId": "HNX-2026-000001"  
}
```

Backend:

Authenticate



Find Registration



Verify ownership



Verify registration status



Set amount = ₹600



Create payment record



Create Karunya transaction

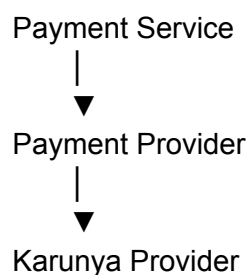


Return payment information

27. Payment Provider Abstraction

Because the exact Karunya payment API has not yet been provided, payment integration should be abstracted.

Architecture:



The application should not have Karunya-specific code scattered throughout registration logic.

28. Payment Provider Interface

Conceptually:

PaymentProvider

```
createPayment()  
verifyPayment()  
handleWebhook()
```

Implementation:

KarunyaPaymentProvider

When Karunya provides its API documentation, only the provider implementation needs to be completed.

29. Payment Response

Example:

```
{  
  "success": true,  
  "data": {  
    "paymentId": "PAY-001",  
    "registrationId": "HNX-2026-000001",  
    "amount": 600,  
    "currency": "INR",  
    "status": "PENDING",  
    "paymentUrl": "..."  
  }  
}
```

The actual response will depend on Karunya's payment mechanism.

30. Payment Page

The frontend should show:

HackNEX Registration

Team:

Code Titans

Participants:

4

Registration Fee:

₹600

Refund Policy:

No Refund

[Proceed to Payment]

After clicking:

Proceed to Payment



Karunya Payment System

31. Payment Verification

Payment verification must happen server-side.

Possible architecture:

Karunya



Webhook



HackNEX Backend



Verify transaction



Verify amount



Verify signature/authentication



Update database

The frontend must not send:

```
{  
  "paymentStatus": "SUCCESS"  
}
```

and expect the backend to trust it.

32. Webhook Endpoint

POST /api/v1/payments/webhook/karunya

This endpoint is intended for the Karunya payment system.

The exact payload will be finalized after obtaining Karunya's API documentation.

33. Webhook Security

The backend should verify whatever Karunya provides, such as:

Webhook signature
Transaction ID
Provider reference
Amount
Currency
Merchant identifier
Timestamp

Conceptually:

Webhook
↓
Signature Valid?
↓
Transaction Valid?
↓
Amount = ₹600?
↓
Already Processed?
↓
YES
↓
Process Payment

34. Payment Amount Verification

Suppose the database contains:

Expected Amount = ₹600

but webhook says:

Amount = ₹500

The payment must not be verified.

Status:

PAYMENT_VERIFICATION_FAILED

This should be logged for administrator investigation.

35. Duplicate Webhooks

Payment providers may send the same webhook more than once.

The system must be idempotent.

Example:

Webhook #1



Payment VERIFIED

Webhook #2



Already VERIFIED



Ignore safely

It must **not**:

- Create another payment
 - Send another confirmation email
 - Create another registration
 - Change confirmed data
-

36. Payment Success Flow

Karunya



Payment Successful



Webhook
↓
Validate
↓
Verify Transaction
↓
Payment = VERIFIED
↓
Registration = CONFIRMED
↓
Team = CONFIRMED
↓
Queue Confirmation Email

37. Confirmation Email Rule

This is a critical business rule:

Registration-success email must be sent only after payment has been successfully verified.

Never:

Registration Created
↓
Confirmation Email ❌

Correct:

Registration Created
↓
Payment Pending
↓
Payment Verified
↓
Registration Confirmed
↓
Confirmation Email ✅

38. Confirmation Email Contents

The email should contain:

HackNEX

Registration Confirmed

Team Name
Registration ID

Captain Name
Captain Email

Participant List

Payment Status
Payment Reference

Event Dates
October 7–9, 2026

Venue
Karunya Institute of Technology and Sciences

Important instructions

The exact email design will be defined in the Email System Specification.

39. Email Failure Handling

Suppose:

Payment verified



Registration confirmed



Resend fails

The registration must **remain confirmed**.

Do not roll back the payment because an email provider failed.

Instead:

Registration = CONFIRMED

Email = FAILED

The email can be retried by a background worker.

40. Payment Failure

If payment fails:

Payment = FAILED

Registration = PAYMENT_PENDING

The user can retry.

Example:

Payment Failed

Your registration has been saved.

[Retry Payment]

41. Payment Retry

The captain should not create another registration to retry payment.

Instead:

Existing Registration



Create/Retry Payment



Karunya

Only one confirmed registration should exist for the captain.

42. Multiple Payment Attempts

The system should maintain payment attempts separately.

Example:

Registration



— Payment Attempt #1 → FAILED



— Payment Attempt #2 → FAILED

|
└─ Payment Attempt #3 → VERIFIED

This gives administrators a complete payment history.

43. Registration Confirmation

Once payment is verified:

Registration.status = CONFIRMED

The team becomes officially registered.

The captain should then see:

Registration Confirmed ✓

44. Post-Confirmation Editing

Once payment is verified, participant information should be considered locked.

CONFIRMED

↓

Participant data locked

This prevents situations where the captain pays for:

Person A

and then replaces them with:

Person B

without administrative control.

If changes are necessary, they should be handled through an admin-controlled process.

45. No Refund Policy

The public registration flow should clearly state:

Registration Fee: ₹600/team

No Refund Policy:

Once payment is successfully completed,
the registration fee is non-refundable.

The exact legal wording should be approved by the organizers.

46. Registration Cancellation

Participants should not have a normal public cancellation/refund API.

There should be no:

POST /payments/refund

for normal users.

If organizers need cancellation capability, it belongs under restricted admin functionality.

47. Registration Ownership

A participant can only access their own registration.

For example:

GET /registrations/:id

must verify:

registration.captainUserId

==

authenticatedUser.id

Otherwise:

403 FORBIDDEN

48. Participant Privacy

A captain should only see the information associated with their own team.

One team must never be able to retrieve another team's:

- Participant information
 - Phone numbers
 - Emails
 - Payment information
-

49. Admin Access

Admins can access:

All registrations
All teams
All participants
All payments

subject to the admin authentication rules.

50. Admin Registration Management

Admin should be able to search:

Registration ID
Team Name
Captain
Participant
Email
College

And filter:

Confirmed
Payment Pending
Payment Failed

51. Admin Payment Management

Admin should be able to view:

Payment ID
Registration ID
Team
Amount
Currency
Status
Transaction ID
Provider Reference
Created At
Verified At

Sensitive provider secrets must never be displayed.

52. Registration Dashboard

The admin dashboard should provide:

Total Teams
Confirmed Teams
Pending Payments
Failed Payments
Total Participants
Confirmed Participants
Total Verified Revenue

For example:

Teams:
375

Participants:
1500

Confirmed:
350

Pending:
20

Failed:
5

Revenue:

₹210,000

These numbers are illustrative.

53. Capacity Consideration

Expected participants:

1,500+

At exactly 4 participants per team:

$1,500 / 4 = 375$ teams

The system should therefore comfortably support at least several hundred teams and thousands of participant records.

The database and API should not be hard-coded to 375 teams.

54. Registration Concurrency

Multiple users may register simultaneously.

Example:

User A → Team Alpha

User B → Team Beta

User C → Team Gamma

...

The backend must safely handle concurrent requests.

Database constraints are required for:

Unique team name

Unique participant email

Unique captain

Application-level checks alone are insufficient.

55. Race Condition Example

Two users simultaneously submit:

Team Name = Code Titans

Both requests may initially pass an application-level check.

The database must ultimately allow only one.

Therefore:

Application Validation

+

Database UNIQUE Constraint

must be used.

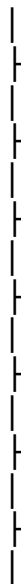
56. Registration API Sequence

React



POST /registrations

Fastify



Authenticate

Verify email

Check registration period

Check captain

Validate request

Check team name

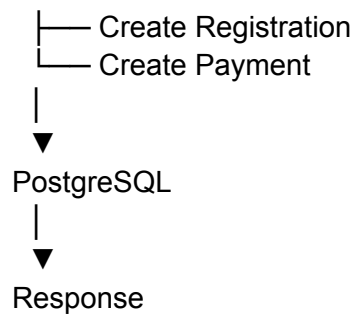
Check participants

Prisma Transaction

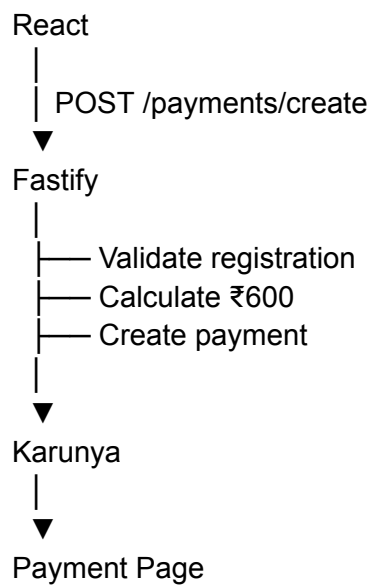


Create Team

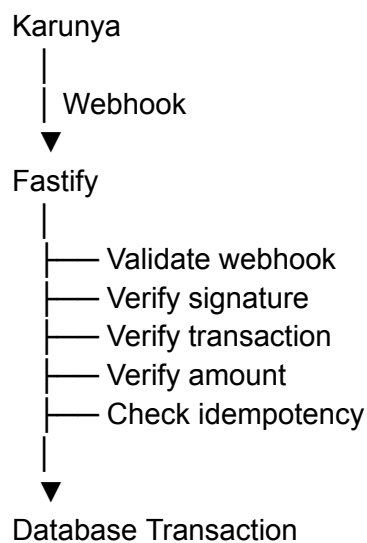
Create Participants

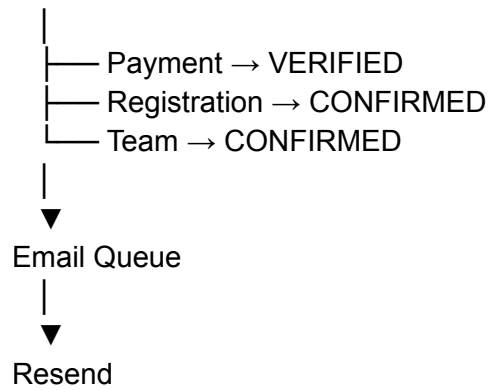


57. Payment Sequence



58. Payment Verification Sequence





59. Failure Scenarios

The system must explicitly handle:

User not verified

→ Registration rejected

Registration period closed

→ Registration rejected

Captain already registered

→ Registration rejected

Duplicate team name

→ 409 Conflict

Duplicate participant

→ 409 Conflict

Invalid participant information

→ 422 Validation Error

Payment failure

→ Payment FAILED

→ Registration remains PAYMENT_PENDING

Invalid payment webhook

→ Reject webhook

→ Do not confirm registration

Wrong payment amount

→ Reject verification

Duplicate webhook

→ Ignore safely

Email delivery failure

→ Registration remains CONFIRMED

→ Retry email

60. Security Requirements

The registration/payment module must implement:

HTTPS

Authentication

Authorization

Input validation

Rate limiting

Secure sessions

Password hashing

CSRF protection where applicable

CORS restrictions

Webhook verification

Database constraints

Transaction handling

Idempotency

Audit logging

Secret management

61. Rate Limiting

Particularly protect:

POST /auth/login

POST /auth/signup

POST /auth/resend-verification

POST /auth/forgot-password

POST /registrations

POST /payments/create

Payment creation should have additional protection against repeated automated requests.

62. Idempotency

Payment operations should support idempotency.

For example:

Request

Idempotency-Key: abc123

If the same request is submitted twice:

First request → creates payment

Second request → returns existing payment

rather than creating duplicate transactions.

63. Audit Trail

Important events should be recorded.

Examples:

REGISTRATION_CREATED

REGISTRATION_SUBMITTED

PAYMENT_CREATED

PAYMENT_PROCESSING

PAYMENT_FAILED

PAYMENT_VERIFIED

REGISTRATION_CONFIRMED

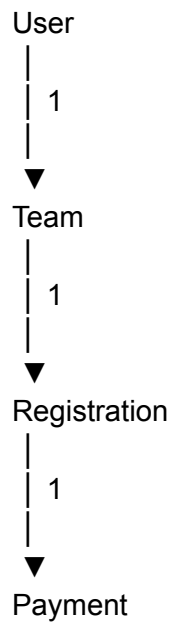
CONFIRMATION_EMAIL_SENT

CONFIRMATION_EMAIL_FAILED

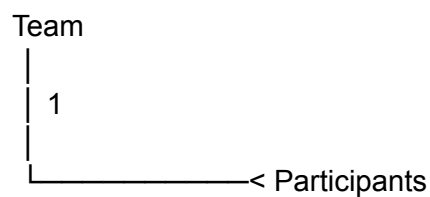
This is extremely useful when handling participant support issues.

64. Recommended Database Relationships

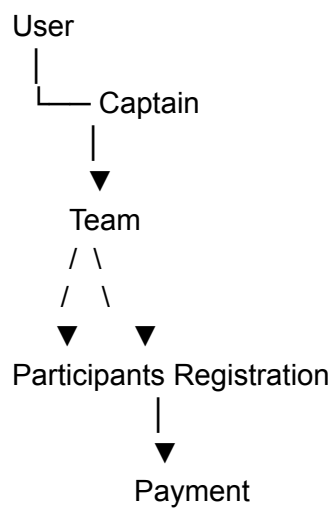
The logical relationship is:



And:



Conceptually:



65. Important Database Constraints

The database should enforce:

User.email UNIQUE

Team.name UNIQUE

Team.captainUserId UNIQUE

Participant.email UNIQUE

Payment.providerTransactionId UNIQUE

Exact constraint names will be finalized in the Database Design Document/Prisma schema.

66. Registration Locking

After:

Payment = VERIFIED

the following should become immutable for normal users:

Team

Participants

Payment amount

Payment reference

Registration confirmation

Admin-controlled modifications can be implemented separately if required.

67. User Experience After Payment

After successful payment:

Payment Successful ✓

Registration Confirmed ✓

Team:

Code Titans

Registration ID:

HNX-2026-000001

Confirmation email has been sent to:
captain@example.com

68. What the Participant Does After Registration

The HackNEX website is **not a continuing participant portal**.

After successful registration:

Registration
↓
Confirmation Email
↓
Future information
↓
Registered Email

Further event communication will primarily happen through email.

The website does not need a complex participant dashboard.

69. Participant Dashboard

A lightweight dashboard is still recommended after login:

Welcome, Infant

Registration Status:

✓ Confirmed

Team:

Code Titans

Registration ID:

HNX-2026-000001

Payment:

✓ Verified

If payment is pending:

Registration:
Payment Pending

[Complete Payment]

This is useful for users who leave the payment flow and return later.

70. Final Registration State Model

Account

UNVERIFIED



ACTIVE



SUSPENDED / DISABLED

Registration

DRAFT



PAYMENT_PENDING



CONFIRMED

Payment

PENDING



PROCESSING



→ FAILED



→ SUCCESS



VERIFIED

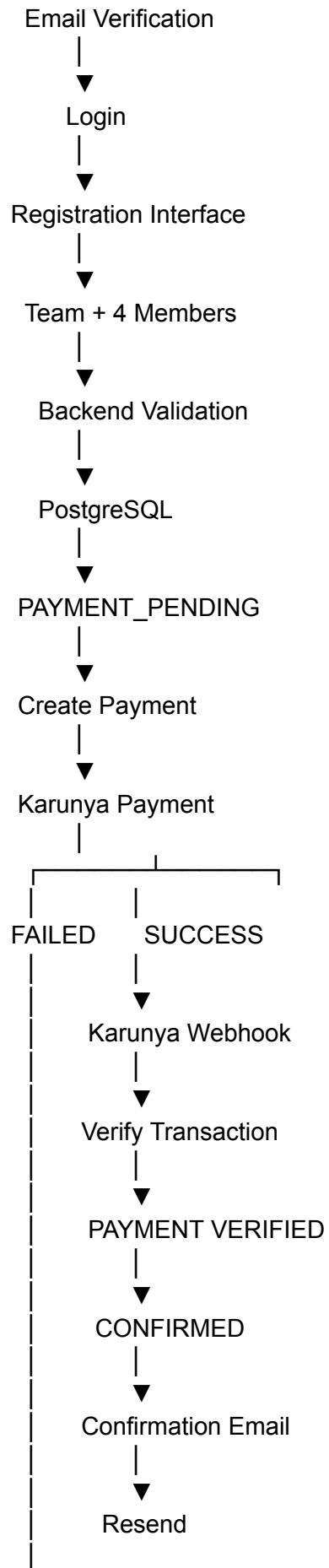
71. Complete End-to-End Architecture

HACKNEX



Signup/Login





72. Final API Requirements

The registration/payment implementation must provide at minimum:

POST /api/v1/registrations
GET /api/v1/registrations/me
GET /api/v1/registrations/:id
PATCH /api/v1/registrations/:id
POST /api/v1/registrations/:id/submit

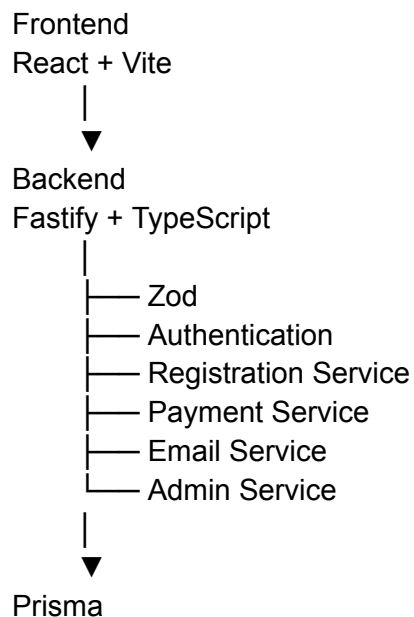
POST /api/v1/payments/create
GET /api/v1/payments/:id
POST /api/v1/payments/webhook/karunya

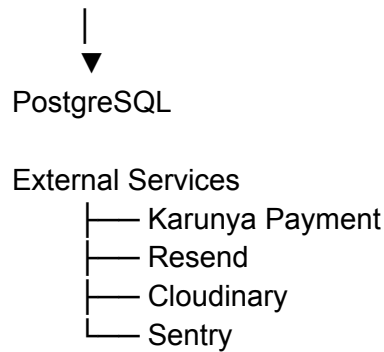
Admin:

GET /api/v1/admin/registrations
GET /api/v1/admin/registrations/:id
GET /api/v1/admin/payments
GET /api/v1/admin/payments/:id

73. Technology Components

The implementation will use:





74. Critical Design Decision

The most important architectural principle for HackNEX is:

The frontend never decides whether a registration is successful.

The frontend can display:

Payment Successful

but the backend must independently establish:

Payment verified

↓
Registration confirmed

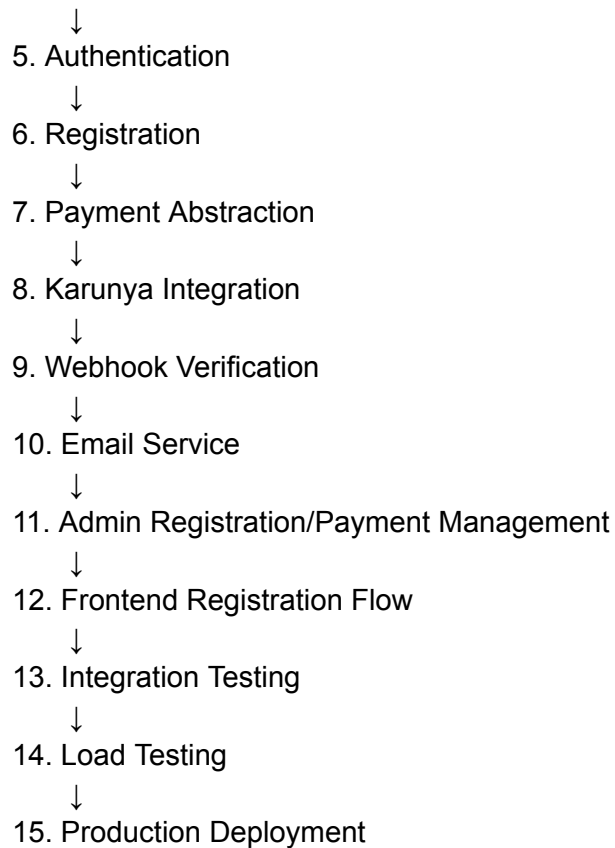
Only then should the registration-success email be queued.

This prevents a malicious user from manipulating frontend requests to obtain a false registration confirmation.

75. Implementation Priority

Once the authentication design and this registration/payment design are finalized, the implementation should proceed in this order:

1. Project Setup
- ↓
2. PostgreSQL
- ↓
3. Prisma Schema
- ↓
4. Fastify Backend



One thing is still intentionally TBD

The **Karunya payment API specification** is the only external dependency that cannot be finalized from the information currently available.

Once you obtain from Karunya:

- Payment API documentation
- Sandbox/test environment
- Merchant/account setup requirements
- Payment initiation API
- Callback/webhook format
- Transaction verification API
- Signature/hash mechanism
- Success/failure status codes
- Test credentials/configuration

the `KarunyaPaymentProvider` and webhook section can be filled in precisely.

Everything else in this document is ready to serve as the implementation blueprint.